

Name:Yazhini R

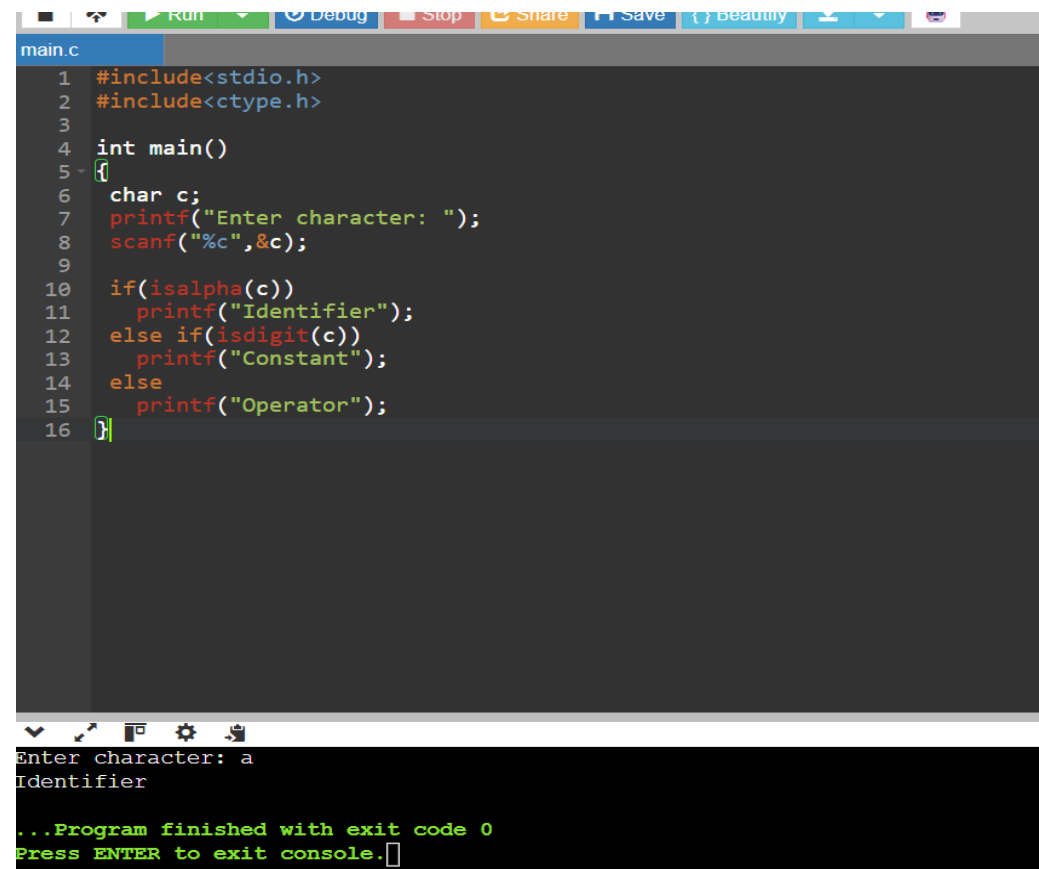
Reg No:192521190

CSA1403-COMPILER DESIGN

LIST OF EXPERIMENTS

1.The lexical analyzer should ignore redundant spaces, tabs, and new lines. It should also ignore comments.

Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Develop a lexical Analyzer to identify identifiers, constants, and operators using the C program.



```
main.c
1  #include<stdio.h>
2  #include<ctype.h>
3
4  int main()
5  {
6      char c;
7      printf("Enter character: ");
8      scanf("%c",&c);
9
10     if(isalpha(c))
11         printf("Identifier");
12     else if(isdigit(c))
13         printf("Constant");
14     else
15         printf("Operator");
16 }
```

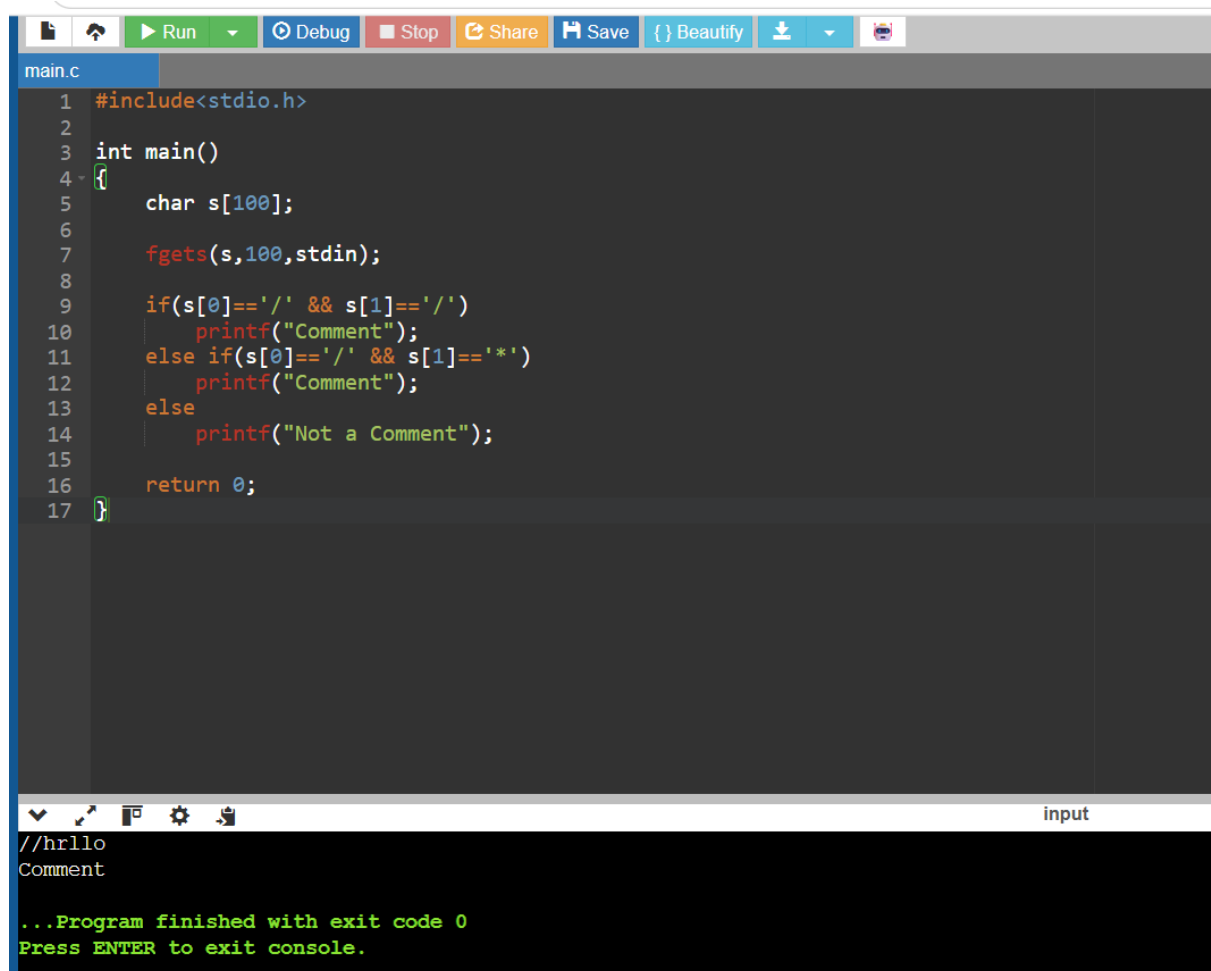
Enter character: a
Identifier
...Program finished with exit code 0
Press ENTER to exit console.

2. Extend the lexical Analyzer to Check comments, defined as follows in C:

a) A comment begins with `//` and includes all characters until the end of that line.

b) A comment begins with `/*` and includes all characters through the next occurrence of the character sequence

`*/`. Develop a lexical Analyzer to identify whether a given line is a comment or not.



The screenshot shows a code editor with a file named `main.c`. The code is as follows:

```
1 #include<stdio.h>
2
3 int main()
4 {
5     char s[100];
6
7     fgets(s,100,stdin);
8
9     if(s[0]=='/' && s[1]=='/')
10         printf("Comment");
11     else if(s[0]=='/' && s[1]=='*')
12         printf("Comment");
13     else
14         printf("Not a Comment");
15
16     return 0;
17 }
```

Below the code editor, the execution output is shown. The input `//hrlllo` is provided, and the program outputs `Comment`. The message `...Program finished with exit code 0` and `Press ENTER to exit console.` is displayed at the bottom.



The screenshot shows a code editor with a file named `main.c`. The code is as follows:

```
1 #include<stdio.h>
2
3 int main()
4 {
5     char s[100];
6
7     fgets(s,100,stdin);
8
9     if(s[0]=='/' && s[1]=='/')
10         printf("Comment");
11     else if(s[0]=='/' && s[1]=='*')
12         printf("Comment");
13     else
14         printf("Not a Comment");
15
16     return 0;
17 }
```

Below the code editor, the execution output is shown. The input `/*hello*/` is provided, and the program outputs `Comment`. The message `...Program finished with exit code 0` and `Press ENTER to exit console.` is displayed at the bottom.

```
main.c
1  #include<stdio.h>
2
3  int main()
4  {
5      char s[100];
6
7      fgets(s,100,stdin);
8
9      if(s[0]=='/' && s[1]=='/')
10         printf("Comment");
11     else if(s[0]=='/' && s[1]=='*')
12         printf("Comment");
13     else
14         printf("Not a Comment");
15
16     return 0;
17 }
```

```
int a=10
Not a Comment

...Program finished with exit code 0
Press ENTER to exit console.
```

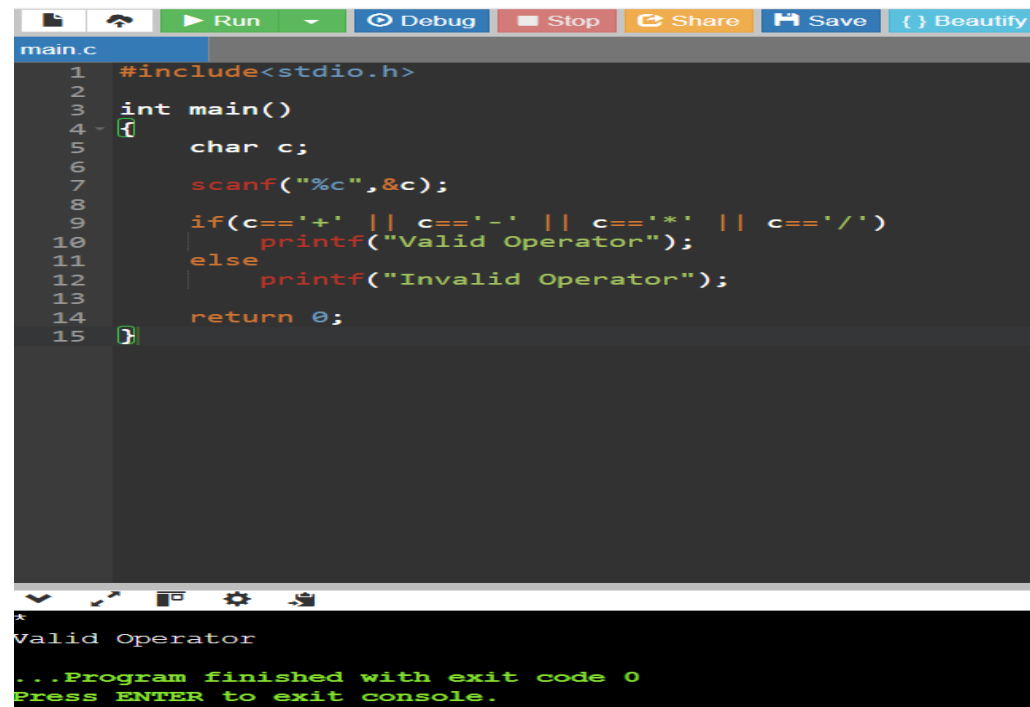
3.Design a lexical Analyzer for a given language should ignore the redundant spaces, tabs, and new lines and ignore comments.

```
main.c
1  #include<stdio.h>
2
3  int main()
4  {
5      char s[100];
6      int i;
7
8      fgets(s,100,stdin);
9
10     for(i=0;s[i]!='\0';i++)
11     {
12         if(s[i]!=' ' && s[i]!='\t' && s[i]!='\n')
13             printf("%c",s[i]);
14     }
15
16     return 0;
17 }
```

```
hello world
helloworld

...Program finished with exit code 0
Press ENTER to exit console.
```

4. Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular Arithmetic operators.

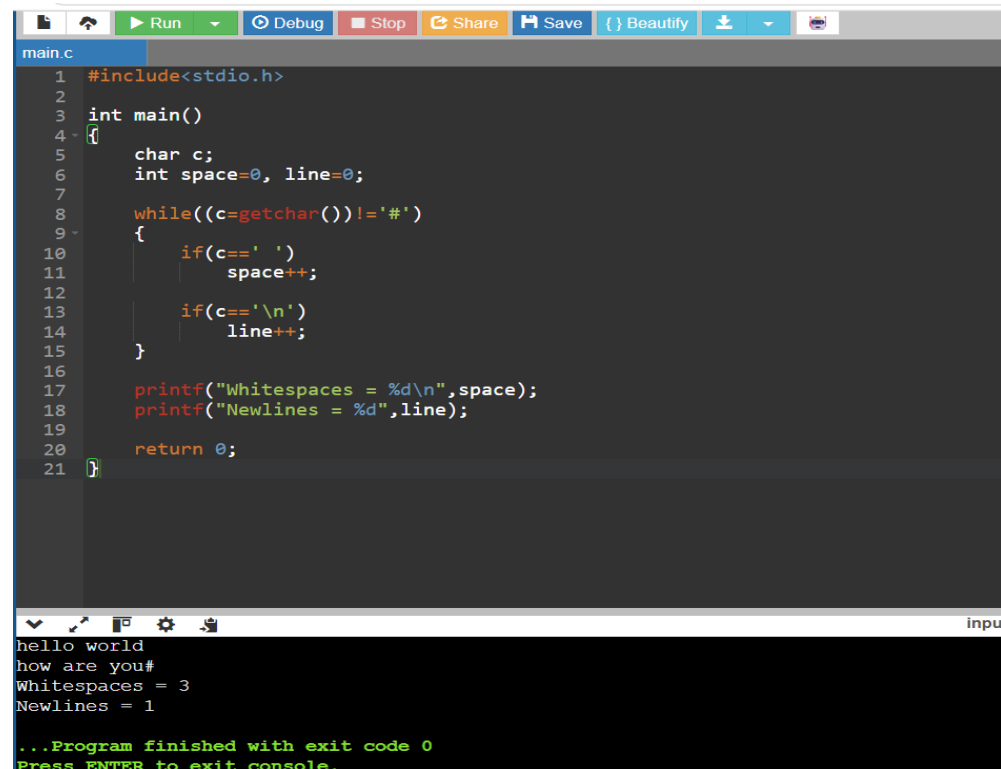


The screenshot shows a C program in an IDE. The code is as follows:

```
1 #include<stdio.h>
2
3 int main()
4 {
5     char c;
6
7     scanf("%c",&c);
8
9     if(c=='+' || c=='-' || c=='*' || c=='/')
10        printf("Valid Operator");
11    else
12        printf("Invalid Operator");
13
14    return 0;
15 }
```

The console output shows "Valid Operator" and "...Program finished with exit code 0 Press ENTER to exit console."

5. Design a lexical Analyzer to find the number of whitespaces and newline characters.



The screenshot shows a C program in an IDE. The code is as follows:

```
1 #include<stdio.h>
2
3 int main()
4 {
5     char c;
6     int space=0, line=0;
7
8     while((c=getchar())!='#')
9     {
10         if(c==' ')
11             space++;
12
13         if(c=='\n')
14             line++;
15     }
16
17     printf("Whitespaces = %d\n",space);
18     printf("Newlines = %d",line);
19
20     return 0;
21 }
```

The console output shows the input "hello world\nhow are you#", followed by "Whitespaces = 3" and "Newlines = 1", and finally "...Program finished with exit code 0 Press ENTER to exit console."

